



LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Zephyrからコードのベアメタル版を抽出してRTOSで使うにはどうすればいいですか？

How to extract a bare metal flavor of code out of Zephyr to use in RTOS ?

Khasim Syed Mohammed

khasim@ti.com

Texas Instruments

Amneesh Singh

Soumya Tripathy

Andrew Davis

11th Dec 2025

Who we are - <https://ti.com/opensource>

- Our quality publicly available documentation allows for for inovation, TI in Zephyr from the start:
 - `git show $(git rev-list --max-parents=0 origin/main) --stat | grep ti_lm3s6965`
- Upstream Zephyr support for devices accross the company
 - **Processors** BU (J721e, AM64, AM62, AM62L, ..)
 - **Application Specific Microcontrollers** (AM243x, AM275x, ..)
 - **MSP** (MSP432P4x, MSPM0Gx, MSPM0Lx, ..)
 - **SimpleLink - Connectivity** (CC13x, CC26x, CC32x, ..)
 - And many more to be supported as upstreaming continues!



Decades of contribution and collaboration

Upstream FIRST mentality!

Upstream and opensource
ecosystem in device architecture

Focus on long term,
sustainable and quality
products



Open
Source

Scope

- Zephyr RTOS offers a rich ecosystem, but embedded engineers often face the challenge of porting code across environments—from Zephyr to another RTOS or even to bare metal.
 - This discussion is about a practical guide to extracting a “bare metal flavor” of code out of Zephyr so that it can run independently of Zephyr’s driver and subsystem layers.
 - NOTE: The HAL approach isn't the right solution, it is making the code stay out of common code base, not following Zephyr standards or coding principles, not gaining advantages of opensource community for reviews of design architecture and the source code.
- We will explore:
 - Structuring Zephyr projects so application and peripheral code can be isolated cleanly
 - Writing drivers and hardware access layers that can compile both inside and outside Zephyr
 - Techniques for minimizing dependencies on Zephyr APIs
 - A case study: extracting a Zephyr-based peripheral driver and reusing it in a bare-metal project
 - Beyond the mechanics, we’ll discuss the philosophy of portability—how to strike the right balance between leveraging Zephyr abstractions and keeping code flexible enough to live elsewhere.

RTOS Usage Scenarios

Processors

General Purpose
Micro
Controller

- RTOS on General Purpose MCUs for real time use cases

Multi-core
Micro
Processor

- RTOS on Core 0 or All Cores
- Linux on Core 0 | RTOS on other Cores
- Linux and RTOS with Hypervisor Multi-cores

Multi-core
Micro
Processor

MCU

- RTOS on Core 0 or All Cores
- Linux on Core 0 | RTOS on other Cores
- RTOS on General purpose MCU to avoid MPU intervention

Multi-core
Micro
Processor

MCU - H/W
Accelerator

MCU

- Linux on Core 0 | RTOS on other Cores
- Linux and RTOS with Hypervisor Multi-cores
- RTOS for MCU controlled hardware accelerators

Applications

Time Critical

Safety Driven

Low Power

Secure

Connected

Quick boot

Ease of use

Scalable RTOS Development Flow across the Industry

Productized, bug fixes & CVE management



Path to **safety compliance**, migrate to a qualified OS (commercial or open source)



Safety RTOS



Look out for **framework** and software stack (Networking, USB, PCIe, Flash, etc.)

Example: FreeRTOS-Plus-TCP, FreeRTOS-Plus-FAT

does not necessarily meet our production code or testing standards

GUI, File, NET, USB : Stacks

Integrate RTOS (an OS / Scheduler) for multi threaded application, dynamic memory management.

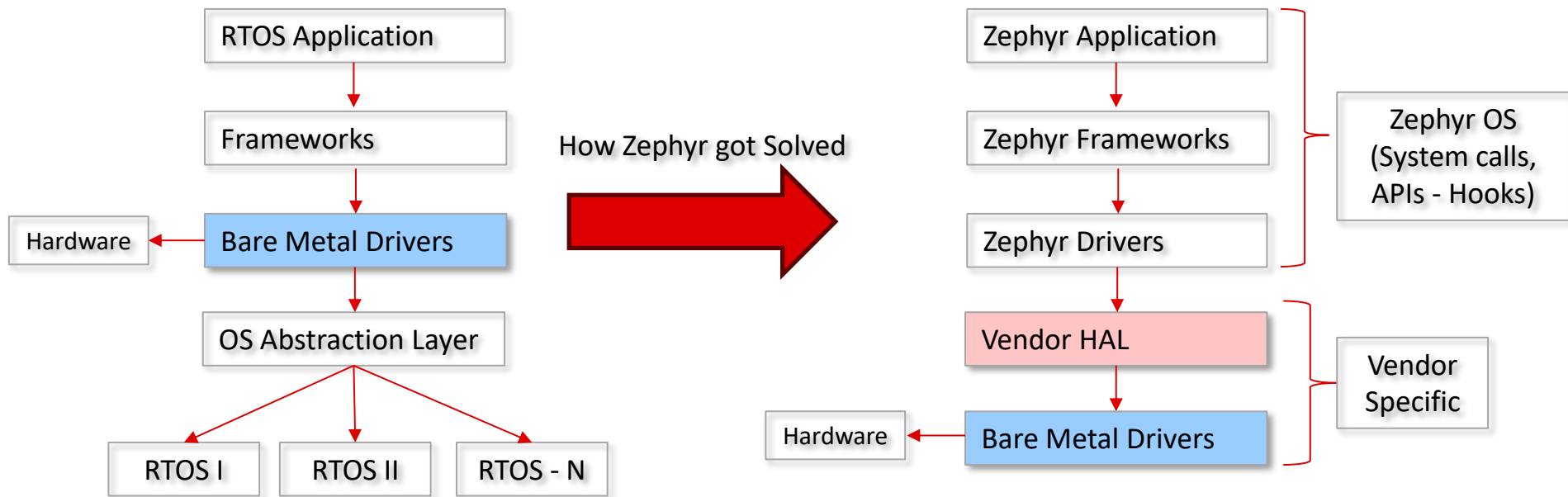


Develop Bare metal drivers, single C file, easy register initialization and debugging

業界はどのようにRTOSの開発を行っていますか？

Current Approach Towards Zephyr

私たちはZephyrを他のRTOSと同じように扱っていますが、それは正しくありません。



- Treating Zephyr as any other variant of RTOS.
- Benefits of open source software stack & community should also be a goto factor.

Why No Zephyr “only” Approach : Industry’s investment (Time & Money)



RTOS	Royalty model	Safety Certified ?	Targeted Cores	SW Stacks - GUI, Networking, USB
FreeRTOS	Free Open Source	No	A53, R5F, DSP	Limited
Eclipse ThreadX	Free Open Source	IEC 61508, IEC 62304, ISO 26262, EN 50128	A cores, R5F, DSP	Available
Zephyr	Free Open Source	In progress	A53, M55, R5, M4	Available
Green Hills	\$ Commercial OS	IEC 61508 SIL4 ISO26262, IEC61508 SIL3	A core	Available
BlackBerry QNX	\$ Commercial OS	IEC61508 SIL3, ISO26262 ASILD, IEC 62304 ClassC, ISO/SAE 21434	A core	Available
SAFERTOS	\$ Commercial OS	IEC 61508-3 SIL 3 ISO 26262 ASIL D	R5F, DSP	Limited



Current Trend

Other RTOS / Bare Metal



Zephyr

New Trend

Zephyr



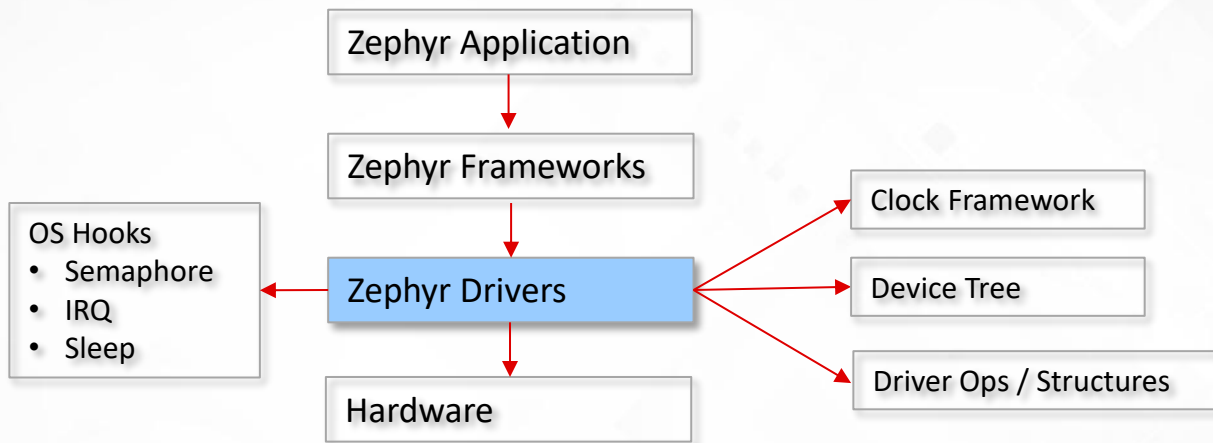
Other RTOS / Bare Metal

Define The Zephyr First Flow .. !

まずは Zephyr First Flow を定義しましょう

- Show a clear path from Zephyr to multiple RTOS
- Leverage Zephyr drivers for Bare metal applications
- Define the best practices for Zephyr driver to make its migration easy.

Detaching The Zephyr Driver for Bare metal usage



- Identify what parts of driver needs bare metal experience
- New owners of pin mux, clock configuration, board configs
- Subsystem and OS call abstractions (Sleep, memory, Sems)

What happens when you try to detach

Is explained in the following slides.

Step 1 : Leverage DTS to the C macros Script

<https://github.com/zephyrproject-rtos/zephyr/tree/main/scripts/dts>

- Device Tree is core of platform configuration.
- It's very easy to migrate the configuration from DT.
- Use script and get macros defined in a header file
- Refer Macros directly in C file
- In General : MCU SDKs of vendors have GUIs for PinMux, map the Macros to make it compatible.

C devicetree_generated.h X

C: > Users > a0393720 > Downloads > C devicetree_generated.h

```
5254 #define DT_N_S_spi_4b00000_P_ti_spi_num_cs_EXISTS 1
5255 #define DT_N_S_spi_4b00000_P_ti_d1_miso_d0_mosi 0
5256 #define DT_N_S_spi_4b00000_P_ti_d1_miso_d0_mosi_EXISTS 1
5257 #define DT_N_S_spi_4b00000_P_clock_frequency 50000000
5258 #define DT_N_S_spi_4b00000_P_clock_frequency_EXISTS 1
5259 #define DT_N_S_spi_4b00000_P_status "disabled"
5260 #define DT_N_S_spi_4b00000_P_status_STRING_UNQUOTED disabled
5261 #define DT_N_S_spi_4b00000_P_status_STRING_TOKEN disabled
5262 #define DT_N_S_spi_4b00000_P_status_STRING_UPPER_TOKEN DISABLED
5263 #define DT_N_S_spi_4b00000_P_status_IDX_0 "disabled"
5264 #define DT_N_S_spi_4b00000_P_status_IDX_0_EXISTS 1
5265 #define DT_N_S_spi_4b00000_P_status_IDX_0_ENUM_IDX 1
```

Step 2 : HLD and LLD

Split the code in High Level Drivers and Low Level Drivers.

- Unlike HAL + Custom Driver, LLD should adhere to Zephyr coding standards.
- Resides on core zephyr github project.
- Reviewed-by community developers and maintainers.
- Tests are developed in Twister.

LLD Examples in Zephyr :

- https://github.com/zephyrproject-rtos/zephyr/blob/53eaf0f71ee9d401b974006c926811f6a3fc4701/drivers/flash/flash_cadence_nand_ll.c
- https://github.com/zephyrproject-rtos/zephyr/blob/53eaf0f71ee9d401b974006c926811f6a3fc4701/drivers/clock_control/clock_agilex_ll.c
- https://github.com/zephyrproject-rtos/zephyr/blob/53eaf0f71ee9d401b974006c926811f6a3fc4701/drivers/clock_control/clock_control_agilex5_ll.c
- https://github.com/zephyrproject-rtos/zephyr/blob/53eaf0f71ee9d401b974006c926811f6a3fc4701/drivers/flash/flash_cadence_qspi_nor_ll.c

Step 3: Manage the sleep

- delay / sleep, play a crucial role in drivers, there are 100+ references for `k_usleep` in zephyr.
- Ensure the functions when ported to another RTOS have similar time delays and effective.
- There are high chances of race conditions and early exists impacting the behavior of the end equipment.
- `k_usleep` is one such example, there are `k_sleep`, `k_timer`, hardware based implementations, etc.

`k_usleep` :

<https://github.com/zephyrproject-rtos/zephyr/blob/main/include/zephyr/kernel.h>

This function is unlikely to work as expected without kernel tuning.

*In particular, because the **lower bound on the duration of a sleep is the duration of a tick**,*

*@kconfig{CONFIG_SYS_CLOCK_TICKS_PER_SEC} must be adjusted to achieve the resolution desired. The implications of doing this must be understood before attempting to use `k_usleep()`. **Use with caution.***

Step 4: Use portable driver patterns: Options:

- **Use struct Definitions for Peripherals**

- Define a struct that maps a peripheral registers
- Alias onto the memory space to access registers using struct dereference
- A widely used and effective style
 - https://github.com/zephyrproject-rtos/zephyr/blob/main/drivers/watchdog/wdt_ti_rti.c

- **Using Pointers and Direct Addressing:**

- Declare a pointer to the register's memory address and simplify dereference it to read or write values.
- Very common in bare metal drivers

- **Use accessor functions**

- Register offset or memory address as argument to function (e.g. readl, ioread32, READ_REG32)
- Provides consistent style between raw MMIO and other access such as SPI/I2C registers (think Linux's regmap)
- Can provide additional safety and debug options at the expense of performance
 - Compiler will usually optimize out the function call over-head by inlining in many simple cases
- Common in Linux drivers and growing in use in Zephyr drivers

Step 5: Avoid non-portable driver patterns

- **Many “thin” RTOSs provide a scheduler and some synchronization primitives**
 - Spawn thread (k_thread_start(), xTaskCreate(), pthread_create())
 - Semaphores, Mutex, Yield/Sleep, Exclusive access data structures
 - Most of these are only syntactically different, safe to use
- **Thin RTOSs often do not provide**
 - Memory management
 - Even when provided tends to be syntactically different
 - Prefer static allocations where possible
 - Complex device management helpers
 - Extensive configuration systems
 - Do not over-depend on KConfig for intra-driver configurability
 - Overuse of #ifdef for code flow control is bad practice anyway

Step 6: Fix up the clock and power management frameworks

<https://github.com/zephyrproject-rtos/zephyr/pull/72102>

- **Avoid explicit management in driver code.**
 - Easier to add to an abstraction to a given RTOS than port clock management code into each and every driver during a port
- **How to manage device power and clocks?**
 - Define static states in DTS
 - Dynamic control only when needed from HLD
- Each RTOS will need to have its own clock framework and management
 - Challenge still exist for bare metal

Step 7: Provide Zephyr First Class in BSP

- **Build tooling around Zephyr drivers not Bare Metal drivers**
 - Sysconfig tools produce DTS suitable for Zephyr, not raw macro headers
- **Examples and demos are Zephyr applications**
 - TI Processor SDK for Linux ships with Zephyr as remote processor firmware
 - <https://git.yoctoproject.org/meta-ti/tree/meta-ti-bsp/recipes-bsp/zephyr-fw>

Step 8: ISR gets OS agnostic

- ISR generally should be easily portable
- Ensure ISR functionality is not handed over to application space.
- Macros and definitions better to keep it around with necessary comments.
- Avoid #ifs to make code simple to port – if multi-SOCs or platforms are thought through.

Step 9: Dev ops (Device structure)

```
#if defined(CONFIG_DEVICE_DEPS) ||
defined(__DOXYGEN__)
    Z_DEVICE_DEPS_CONST device_handle_t
*deps;
#endif /* CONFIG_DEVICE_DEPS */
#if defined(CONFIG_PM_DEVICE) || defined(__DOXYGEN__)
union {
    struct pm_device_base *pm_base;
    struct pm_device *pm;
    struct pm_device_isr *pm_isr;
};
#endif
#if defined(CONFIG_DEVICE_DT_METADATA) ||
defined(__DOXYGEN__)
    const struct device_dt_metadata *dt_meta;
#endif /* CONFIG_DEVICE_DT_METADATA */
```

```
struct device {
    /** Name of the device instance */
    const char *name;
    /** Address of device instance config information */
    const void *config;
    /** Address of the API structure exposed by the device */
    const void *api;
    /** Address of the common device state */
    struct device_state *state;
    /** Address of the device instance private data */
    void *data;
    /** Device operations */
    struct device_ops ops;
    /** Device flags */
    device_flags_t flags;
};
```

Information parsed in this core structure should get a standard understanding, don't make it custom.

Benefits of Zephyr First

- CICD
- Twister : Test Framework for automation
- Robust Framework to test the drivers
- Opensource community based review of the code base
- Complying to CRA for CVE management made easy
 - With 5year LTS
 - Detailed SBOMs

Credits

Thanks to the Zephyr, Linux and Open Source Software Community

khasim@ti.com

Community Partners

